

Module 01: Memory Hierarchy and Address Binding

1. Memory Hierarchy & Tradeoffs

Computer system me CPU speed aur secondary storage ke latency gap ko bridge karne ke liye hierarchy use hoti hai: Registers (<1ns) → Caches (1-10ns) → RAM (50-100ns) → NVMe/SSD (μs) → HDD (ms). Upar se neeche jane par latency badhti hai aur cost per bit ghat ti hai.

2. Bare Machine vs Resident Monitor

Bare Machine: Shuruati systems me zero memory protection tha; user directly hardware ko manipulate karta tha.

Resident Monitor: OS ka first avatar jisme fence register use karke memory ko OS space aur user program space me protect kiya gaya.

3. Logical vs Physical Address Space

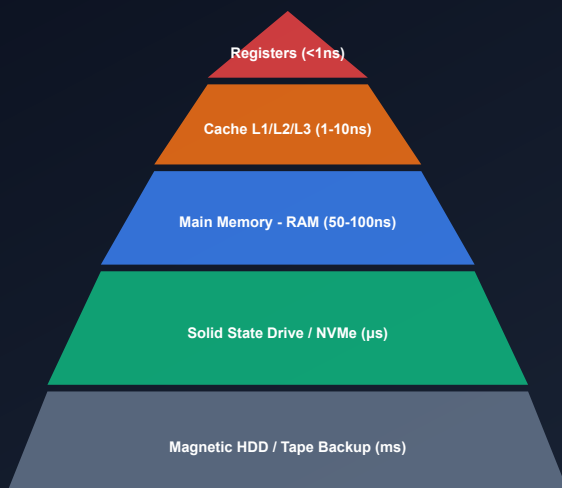
Feature	Logical (Virtual) Address	Physical Address
Generator	CPU execution engine	Memory unit (RAM bus)
User View	0 se MAX tak continuous flat address space	Actual hardware silicon cell address
Translation	MMU dwara runtime par translate hota hai	Directly memory bus par transmit hota hai

4. Address Binding: 3 Phases

- Compile Time:** Absolute addresses generate hote hain. Program relocate nahi ho sakta.
- Load Time:** Relocatable addresses generate hote hain. RAM me load hote hi base address fix ho jata hai.
- Execution (Run) Time:** Dynamic relocation register hardware (MMU) use karta hai. Process runtime par RAM me move ho sakta hai.

Address Binding Phases & Dynamic Relocation

Memory Hierarchy Pyramid



3-Stage Address Binding Flow

1. Compile Time Binding

Generates Absolute Code. Agar starting location badle to recompile karna padta hai.
Example: MS-DOS .COM programs, embedded microcontrollers.



2. Load Time Binding

Generates Relocatable Code. Compiler symbolic offset generate karta hai; Loader memory me load karte waqt final base address bind karta hai.



3. Execution (Run) Time Binding (Modern OS)

Logical addresses runtime par Physical addresses me translate hote hain.

Hardware Support: MMU (Memory Management Unit)

Formula: $\text{Physical Address} = \text{Logical Address} + \text{Relocation Register}$

Allows process to move dynamically in RAM during execution!

5. Dynamic Loading vs Dynamic Linking

Dynamic Loading: Routines tabhi memory me aate hain jab unhe invoke kiya jaye (memory bachti hai).

Dynamic Linking: Shared libraries (.so / .dll) ek baar RAM me load hoti hain aur multiple processes runtime stubs ke through execute karte hain.

6. Solved Numerical

Problem: Base = 14000, Limit = 2500.

1. Logical Address 1240: Check $1240 < 2500$ (Valid). Physical Address = $14000 + 1240 = 15240$.

2. Logical Address 2750: Check $2750 \geq 2500$ (Invalid). Hardware MMU triggers **Trap to OS: Memory Access Violation**.